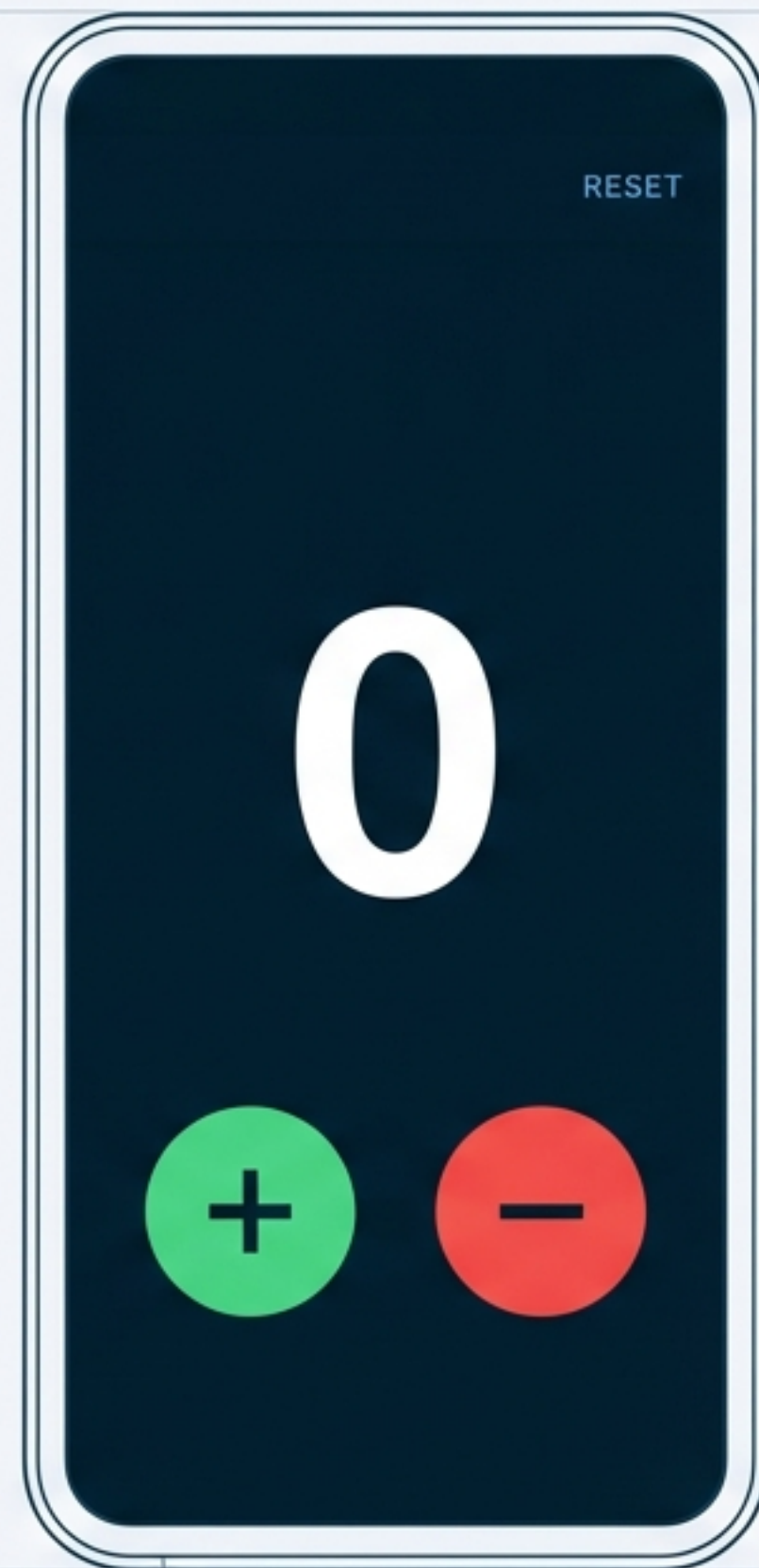


Android アプリ開発入門

カウンターアプリを作って学ぶ
Activity とレイアウトの基礎

Java / Android Studio Panda 1



本日の開発ミッション

シンプルなカウンターアプリの開発を通じて、Android Studio を使用した Java ベースのアプリ開発の基礎を習得します。



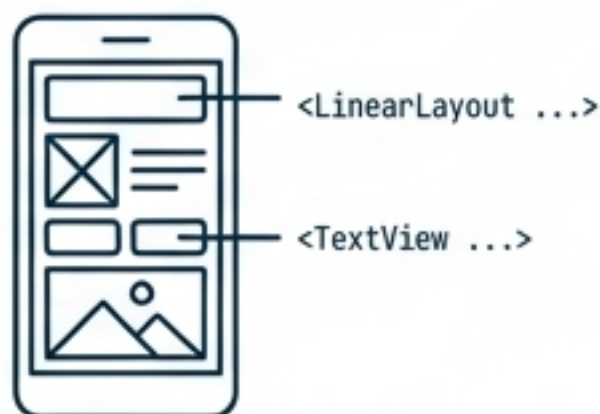
1. プロジェクト構成

ファイルの役割と配置を理解する



2. Activity

ライフサイクルと画面制御を学ぶ



3. XML レイアウト

UI 部品の配置とデザイン定義



```
onClick() {  
    ...  
    count++;  
};
```

4. イベント処理

クリック操作と画面更新の実装

最終的に「UI 配置」「クリック処理」「状態更新」の流れを完全に理解することを目指します。

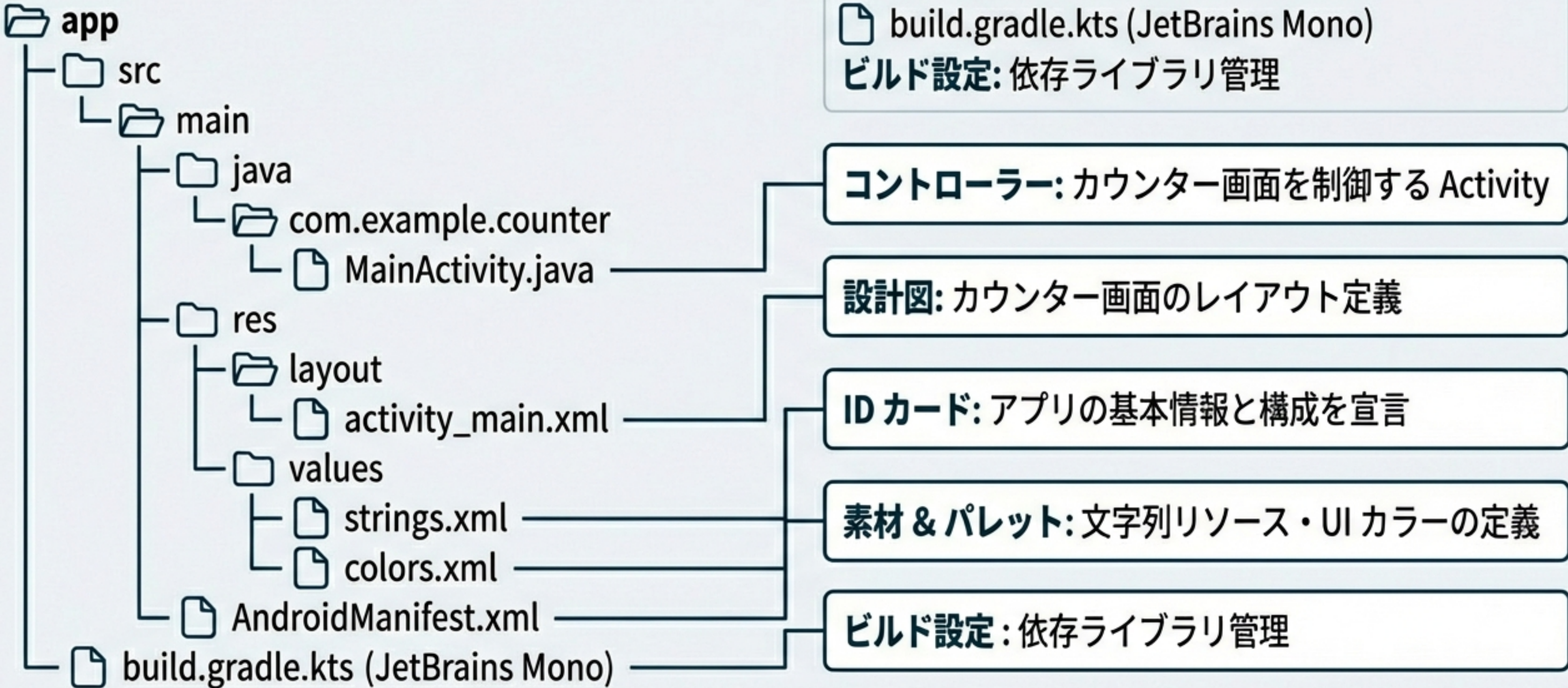
開発環境とツール

項目	バージョン / 詳細
IDE	Android Studio Panda 1 (2025.3.1) 以降
Language	Java (JDK 11)
Minimum SDK	30 (Android 11)
Target SDK	36 (Android 16)



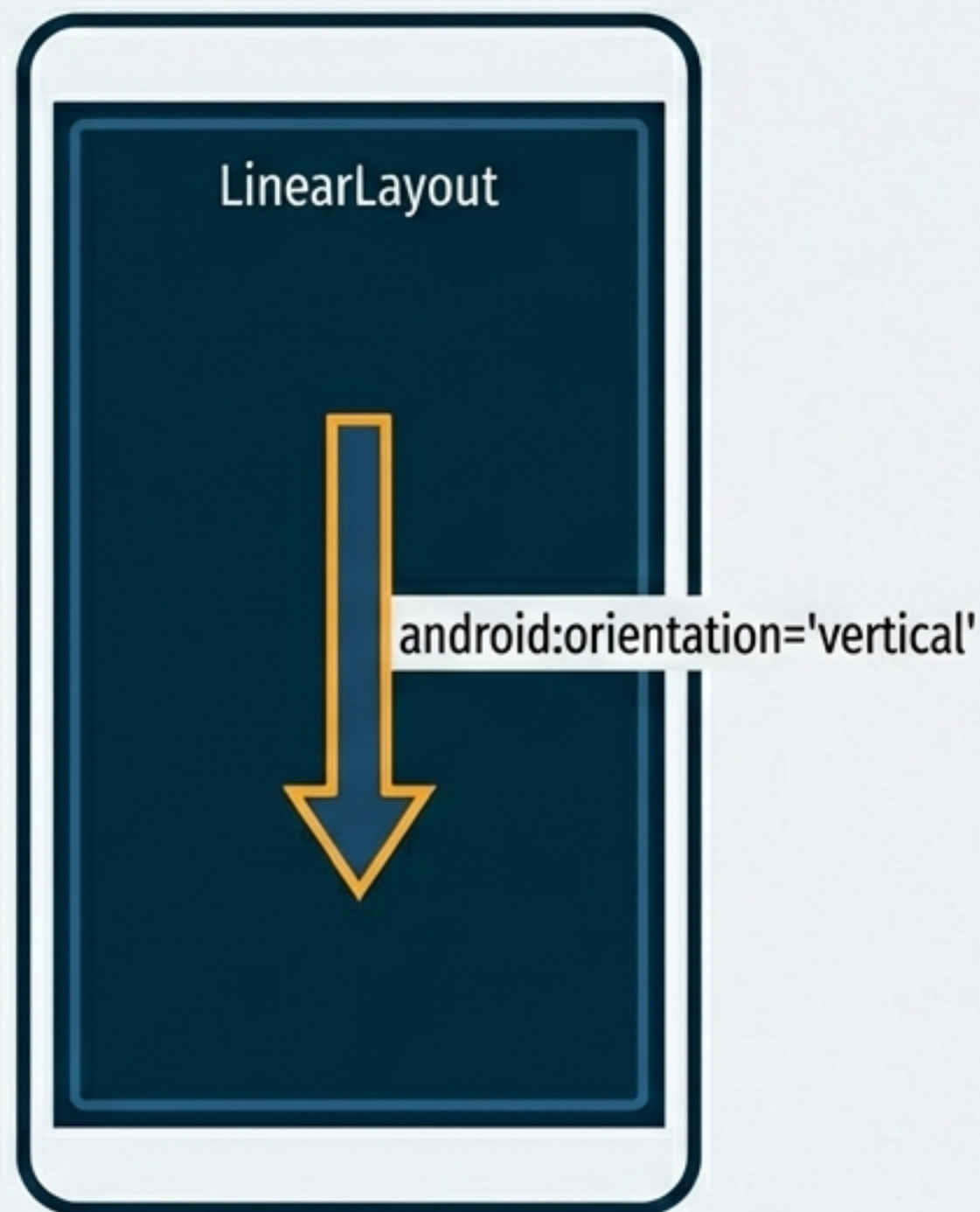
最新の環境設定に基づき、EdgeToEdge などのモダンな実装を含みます。

プロジェクト構成



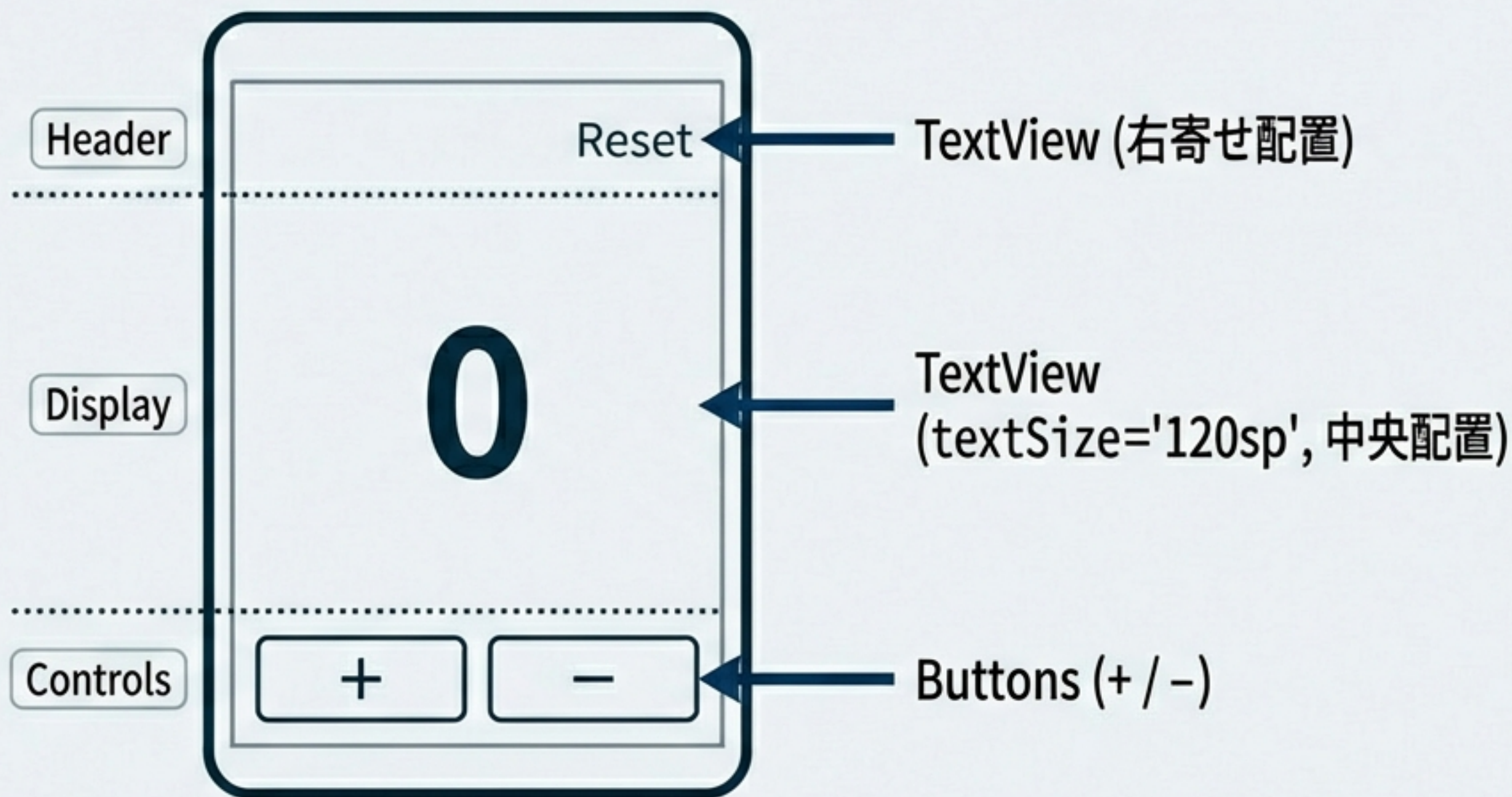
レイアウトの土台

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="..."
    xmlns:tools="..."
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="@color/background_dark">
    <!-- 子要素がここに配置されます -->
</LinearLayout>
```



- XML Declaration: ファイル定義と文字コード
- Namespaces: android: (標準) vs tools: (開発用)
- Root Element: 縦方向に子要素を並べるコンテナ

UI 部品の配置



Header & Display

layout_weight を使用してスペースを調整。
フォントは sans-serif-black で強調。

Footer Controls

layout_weight を使用して横幅を均等または伸縮可能に
制御 (width=0dp)。backgroundTint で色分け。

MainActivity の構造

Engine Room (The Java Class)

```
package com.example.counter;
```

```
import android.os.Bundle;
import android.widget.Button;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;
```

← 必要なパーツ (Tools) の読み込み

```
public class MainActivity extends AppCompatActivity {
```

```
    // フィールド (変数の定義)
```

```
    private int count = 0;
    private TextView txtCount;
```

← データの保持場所 (State)

```
    @Override
```

```
    protected void onCreate(Bundle savedInstanceState) { ... }
```

```
}
```

AppCompatActivity を継承することで、古い Android バージョンでも最新の機能を使えるようにします。

起動と初期化: onCreate

Activity 起動



onCreate() 呼び出し

EdgeToEdge.enable() (全画面表示)
setContentView(R.layout.activity_main) (レイアウト読み込み)

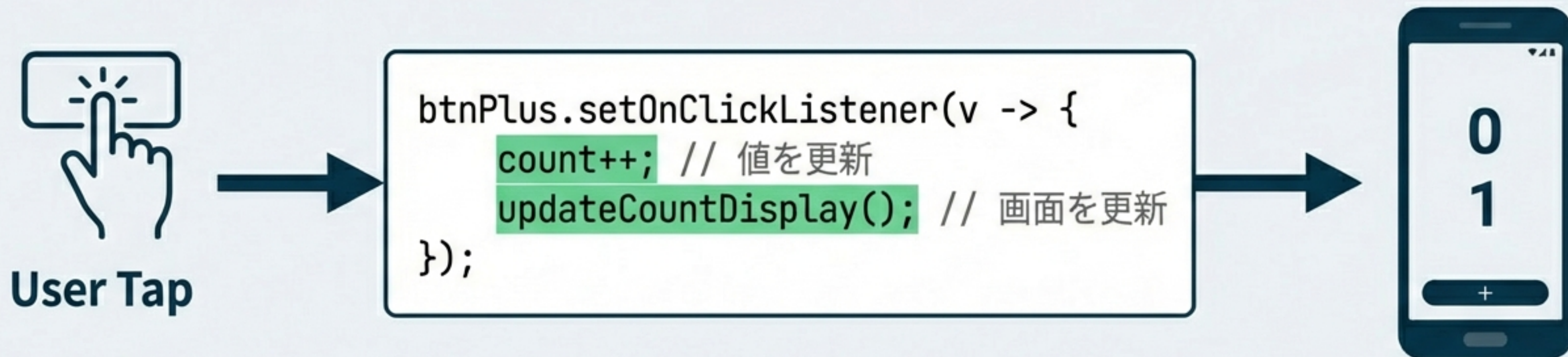


View の取得 (The Bridge)

android:id="@+id/text_count_value"  txtCount = findViewById(R.id.text_count_value);
findViewById

ID を使って、XML 内の部品を Java の変数に接続します。

クリック処理の実装

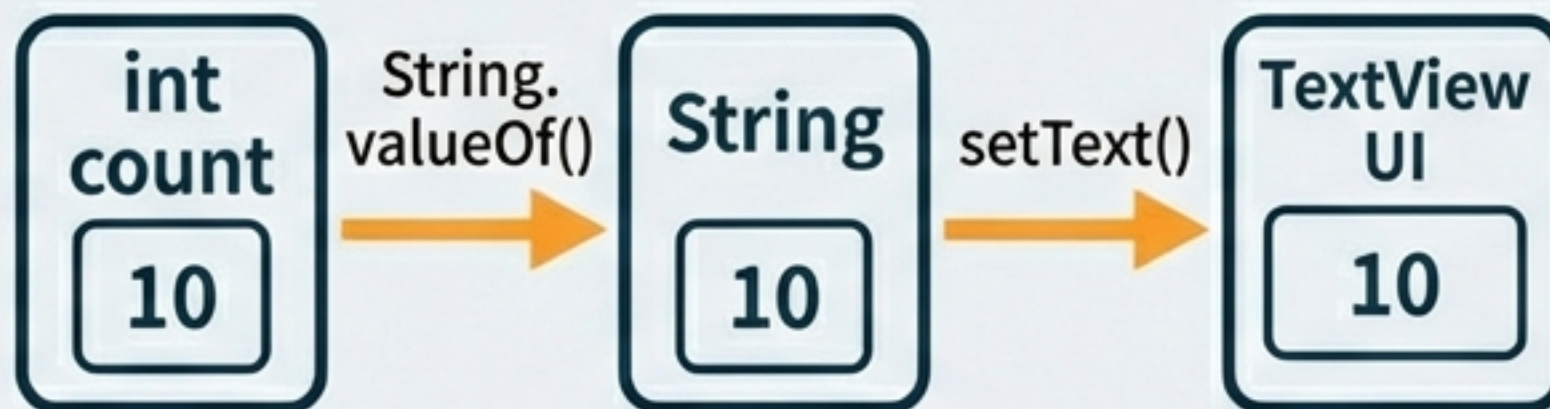


- **setOnClickListener:** ボタンが押された時の動作を定義するリスナー。
- **Logic:** count 変数をインクリメントし、表示更新メソッドを呼び出すことで、データと見た目を同期させます。

表示の更新

ロジックと表示の分離 (Separation of Concerns)

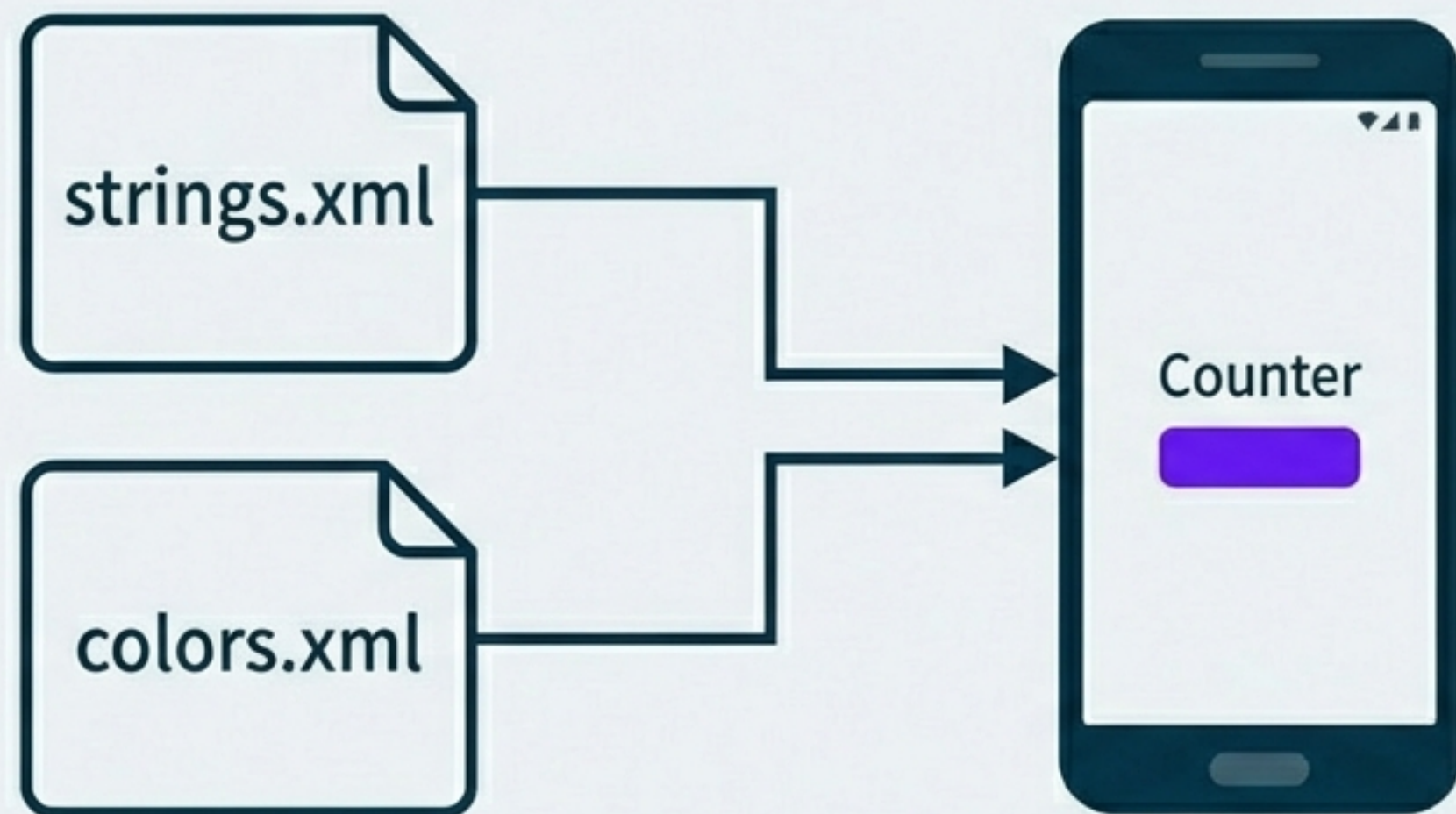
```
private void updateCountDisplay() {  
    txtCount.setText(String.valueOf(count));  
}
```



なぜメソッドを分けるのか？

- **重複コードの排除**: +ボタン、-ボタン、リセットボタンの全てから同じメソッドを呼べる。
- **メンテナンス性**: 表示形式を変更したい場合、この1箇所を修正するだけで済む。

リソースの管理



文言の一元管理・多言語対応

```
<string name='app_name'>Counter</string>
```

カラーパレット定義

```
<color name='primary'>#FF6200EE</color>
```

- **Colors Format:** #AARRGGBB (Alpha, Red, Green, Blue)
- **Usage:** コードに直接書き込まず（ハードコードせず）、リソース ID (@string/..., @color/...) で参照するのが作法です。

アプリの ID カード: AndroidManifest.xml

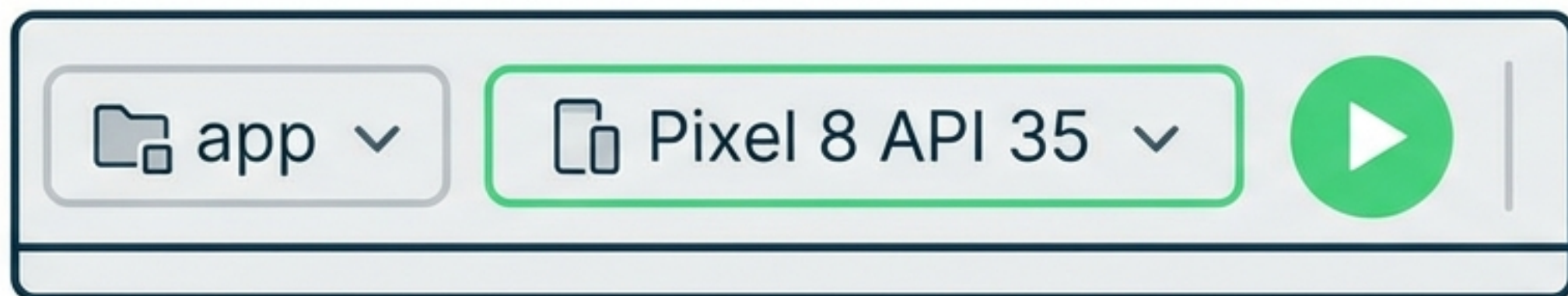


intent-filter:
action.MAIN &
category.LAUNCHER

このアプリは何か」「どの画面から
起動するか」を宣言します。

このファイルは、Android システムに対して「このアプリは何か」「どの画面から起動するか」を宣言します。

ビルドと実行



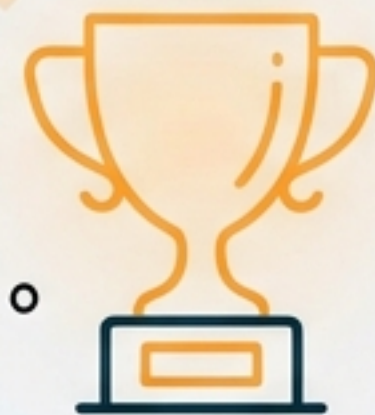
1. プロジェクトを開く
2. エミュレータ/実機を選択
3. Run (▶) ボタンをクリック

Build Successful: APK generated.

ビルドが完了すると、選択したデバイスにアプリがインストールされ、自動的に起動します。

演習問題: Level Up

- ☒ 初期値の表示: onCreate 内で updateCountDisplay() を呼び出し、起動直後に '0' を表示させる。
- ☐ ステップ数の変更: 変動幅を「5」に変更する（定数の利用を推奨）。
- ☐ 負の値ガード: 減算時に 0 未満にならないよう if 文でガード処理を追加する。
- ☐ 画面回転対応: onSaveInstanceState を使用して、画面回転時にカウント値を保持・復元する。



まとめと次のステップ

- **プロジェクト構成:** ディレクトリ構造とファイルの役割
- **Activity:** onCreate ライフサイクルと画面制御
- **Layout XML:** LinearLayout による配置と属性設定
- **イベント処理:** リスナー登録と表示更新ロジック



基礎は完成しました。次はデータの永続化 (SharedPreferences / Database) や、より複雑な UI コンポーネント (RecyclerView) への挑戦が待っています。